

D0_High_Level_Design

AI Workflow Studio

Team: Ashanth Ganesh · Daniel Kreifels · Khiem Ha · Jayrajsinh Gohil

Course: 20CS5001 · **Date:** October 2, 2026

1. Title, goal statement, and conventions.

Goal: Enable users to visually compose reusable AI nodes into provider-independent workflows, execute those workflows reliably, and inspect their outputs, progress, and failures.

Basic input: A user-authored workflow graph with typed node configuration, provider connection references, and runtime input data. **Basic output:** A saved, validated workflow and an execution record containing the resulting data, per-node status, errors, and timing metadata.

D0 boundary: The system covers the browser workspace, access control, workflow management, execution, reusable nodes, provider integration, and persisted application records. Identity providers and AI/external services are dependencies outside the team's control. The first vertical slice uses text/JSON inputs; document, image, audio, and video nodes extend the same contracts later.

Conventions: C1–C7 are logical components the team owns. E1–E2 are external systems, drawn with dashed amber borders. Solid arrows indicate the request or data direction; responses use the same interface in reverse. I1–I13 identify the contracts in section 4. Each diagram includes its own title, goal, and legend. Data-flow boxes represent stages, so a component may appear more than once.

Design status: The five stories in `Design_Diagrams/User_stories.md` cover hosting, placeholder workflow CRUD, database design, CI, and project scaffolding. US1–US5 below are document reference labels, assigned in file order because the source has no IDs. The full editor/execution/provider design is the later product target from `Docs/Project.md`. The component table is accepted for this D0.

Reading guide: Section 2 shows structural boundaries; section 3 assigns responsibilities; section 4 defines every structural connection; section 5 follows access and workflow data; sections 6–7 justify the proposed architecture.

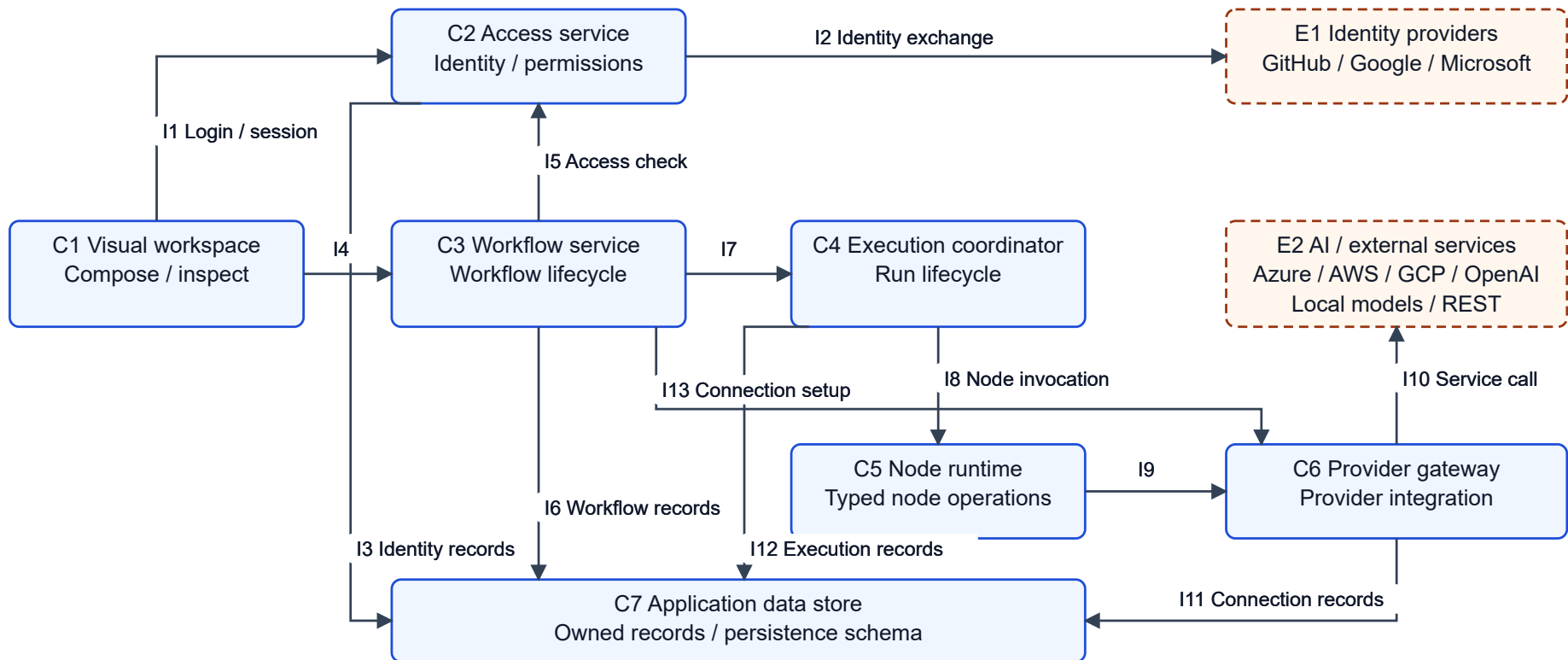
2. Block diagram (the D0 diagram).

AI Workflow Studio — Design D0 | Block diagram

Goal: Visually compose reusable AI nodes, run workflows across providers, and inspect their results.

Legend: solid blue box = team-built component/stage; dashed amber box = external system.

Arrows show request/data direction; replies return over the same interface. Labels identify data or interface IDs.



C2–C6 are logical modules in one backend deployment, not independently deployed services.

C7 is the team-owned data model; its database infrastructure may be local or managed. E1/E2 are outside team control.

Scope: US2 requires placeholder workflow CRUD; C4–C6 describe the later execution target from Project.md.

Seven owned components provide clear boundaries while keeping the backend in one deployable application. Provider choices belong behind C6; workflow execution remains independent of the selected provider. C7 denotes our records and schema, rather than ownership of a managed database vendor.

3. Component responsibility table.

Interfaces carry requests and replies, so each listed ID is both an input and an output boundary. Directional initiators are specified in section 4. The listed owners are primary task owners for Assignment 8.

Component	One-sentence responsibility	Interfaces in / out	Primary owner	Story mapping
C1 Visual workspace	Presents the visual workflow interaction to the user.	I1, I4	Daniel Kreifels	US1, US5: UI foundation / hosting
C2 Access service	Decides whether an identified user may perform a scoped application action.	I1, I2, I3, I5	Khiem Ha	US1, US3: identity configuration / data; auth target in Project.md
C3 Workflow service	Manages the lifecycle of workspace-scoped workflow definitions.	I4, I5, I6, I7, I13	Ashanth Ganesh	US2, US3: workflow CRUD / records
C4 Execution coordinator	Controls the lifecycle of an authorized workflow run.	I7, I8, I12	Ashanth Ganesh	Later target; US3 supports execution schema planning
C5 Node runtime	Evaluates a reusable operation through a common typed node contract.	I8, I9	Jayrajsinh Gohil	Later target built on US5 backend foundation
C6 Provider gateway	Translates generic capability requests into scoped provider interactions.	I9, I10, I11, I13	Khiem Ha	Later target; US1 supplies hosting foundation
C7 Application data store	Persists authoritative application records.	I3, I6, I11, I12	Jayrajsinh Gohil	US2, US3: persisted CRUD / schema

Crosscutting stories: US4 supplies repository health checks, tests, linting, and main-branch protection; US5 supplies scaffolding and documentation. These support the architecture without adding runtime arrows. C4–C6 are future product components, not features promised by the five foundation stories.

External dependencies: E1 authenticates a human; E2 performs an AI, REST, or local-model capability. Azure hosting is a deployment dependency, separate from E2's workflow service calls.

3. Component responsibility table. (traceability)

Traceability to User_stories.md

US1–US5 are reference labels assigned by this document in source order, not IDs supplied by the team. Original titles are preserved below; the required work is summarized without inventing end-user workflow stories.

Ref.	Original story title	Required work / stated use case	Component mapping / architectural effect
US1	Configure Azure environment	Create developer Azure accounts, a resource group, API/UI app registrations; configure Static Web Apps for UI and App Service for API; optionally use Key Vault for OAuth credentials. Provide a deployed full application for end-to-end testing.	C1 UI hosting; C2 identity configuration; C2–C6 hosted backend boundary. Hosting is deployment work, not an additional runtime module. App Service vs Container Apps discrepancy is recorded in section 6.
US2	Create Workflow CRUD Api	Provide placeholder API CRUD operations against the database using a temporary workflow object, enabling later attributes and validation.	C3 workflow lifecycle, C7 persistence; I4 covers create/read/update/delete and I6 covers stored records. A full graph editor or execution engine is not required by this story.
US3	Design Database Architecture	Identify relational tables, fields, foreign keys, and 1:1 / 1:M / M:1 / M:M relationships.	C7 owns the data model; C2/C3/C4/C6 identify their record needs. Target relationships: user→identities/sessions 1:M; user↔workspace M:M via membership; workspace→workflow/connection 1:M; workflow→execution→node result 1:M.
US4	Setup CI pipeline	Automate repository health checks, tests, and linting to protect main and ensure project integrity.	Crosscuts C1–C7 through development checks. CI definitions live in .github/workflows; branch-protection/required-check settings are separate repository configuration. It does not add a production runtime connection.
US5	Setup project documentation and platform fundamentals	Set up Angular or React using Vite, a FastAPI project with a configured virtual environment, and a documentation folder with at least one Markdown file.	C1 frontend foundation; C2–C6 backend foundation; repository documentation records C7 schema and system contracts. The existing React scaffold is consistent with the story's allowed choices.

Scope distinction: All five supplied stories are mapped. The full node/execution/provider pipeline is retained as the product target from Project.md, with only indirect foundation support from these stories. Dedicated execution/editor/provider stories remain to be added by the team.

4. Interface specification table.

Each structural arrow appears exactly once below. Input means the initiating request; output means the reply. These are D0 contracts; endpoint naming and implementation details belong in D1/D2.

ID / request direction	Named, typed inputs	Named, typed outputs	Data format	Protocol	Error behavior / responsible component
I1 · C1 → C2	provider:string, return_to:path; session_cookie:opaque string; csrf_token:string for logout.	redirect_url:URL or user:User; Set-Cookie:opaque session/CSRF; logout:empty.	JSON for discovery/session; query parameters and cookie/header fields.	HTTP locally; HTTPS hosted. Browser GET sign-in/session; POST logout.	C2 rejects absent/expired session (401), invalid CSRF (403), or invalid callback; C1 shows a safe sign-in error.
I2 · C2 → E1	client_id:string, redirect_uri:URL, state:string, scope:string, pkce_challenge:string; callback code:string; server-side verifier:string.	authorization_code:string, state:string; profile: {subject:string,email:string,display_name:string?,avatar_url:URL?}.	Authorization query; form-encoded token exchange; JSON identity profile.	HTTPS OAuth/OIDC authorization-code exchange; browser-mediated redirects.	C2 validates one-time state/browser binding/expiry, rejects cancelled or invalid login, normalizes upstream failures; no session issued.
I3 · C2 → C7	identity: {provider:string,subject:string,email:string}; session: {user_id:UUID,token_digest:string,csrf_digest:string,expires_at:timestamp}; OAuth attempt / membership lookup.	user:User; session_valid:bool; permission_membership: {workspace_id:UUID,role:string?}; commit_ok:bool.	Parameterized typed records; timestamps; persisted relational rows.	Database request/reply over SQL protocol; TLS when hosted.	C7 rolls back failed writes; C2 fails closed on unavailable storage (503) and never accepts an unverified session.
I4 · C1 → C3	workspace_id:UUID; workflow:WorkflowDraft?; workflow_id:UUID?; inputs:ValueMap? for run; execution_id:UUID? for status; session/CSRF:string.	workflow:WorkflowRecord or workflows:WorkflowRecord[]; deleted:bool; errors:Error[]; execution: {id:UUID,status:RunStatus,node_results:NodeResult[]}.	JSON body/reply; IDs in path/query; session cookie / CSRF header on writes.	HTTP locally / HTTPS hosted REST. CRUD: POST, GET, PUT/PATCH, DELETE. Later run POST / status GET.	C3 returns 401/403, 404 missing record, 422 field errors, 409 conflict, 503 storage/runtime failure. C1 preserves failed edits; deletion success requires committed removal.
I5 · C3 → C2	session_token:opaque string, csrf_token:string?; workspace_id:UUID; action:read write execute manage_connection; resource_id:UUID?.	access: {allowed:bool,user_id:UUID,workspace_id:UUID,role:string}; error:Error?.	Typed in-process request/reply; no provider credential fields.	Internal application call within the backend.	C2 denies missing session, missing membership, bad CSRF, or insufficient role; C3 maps denial to 401/403 before side effects.
I6 · C3 → C7	workspace_id:UUID; workflow_id:UUID?; workflow:WorkflowDraft?; operation:create read update delete list.	workflow:WorkflowRecord or workflows:WorkflowRecord[]; deleted:bool; commit_ok:bool.	Typed relational records; optional versioned graph JSON for later attributes.	Parameterized SQL request/reply; TLS hosted.	C7 rolls back failed writes/deletes and reports conflict/unavailable; C3 returns 409/503. Referenced records cannot be removed contrary to the chosen foreign-key policy.

4. Interface specification table. (continued)

Each structural arrow appears exactly once below. Input means the initiating request; output means the reply. These are D0 contracts; endpoint naming and implementation details belong in D1/D2.

ID / request direction	Named, typed inputs	Named, typed outputs	Data format	Protocol	Error behavior / responsible component
I7 · C3 → C4	action:start status; authorized_scope: {user_id:UUID,workspace_id:UUID}; graph_snapshot:Graph + inputs:ValueMap for start; execution_id:UUID for status.	execution_id:UUID; status:RunStatus; node_results:NodeResult[]; error:Error?.	Typed in-process records; JSON-compatible values.	Internal application call; start acknowledges a persisted run; status reads execution state.	C4 rejects unsupported/invalid run; persistence failure prevents acceptance. C3 maps failures to 422/503; interrupted runs are surfaced as failed/interrupted.
I8 · C4 → C5	execution_id:UUID, node_id:string, node_type:string, config:object, inputs:ValueMap, authorized_scope:object, timeout_ms:integer.	node_result: {node_id:string,status:RunStatus,outputs:ValueMap,latency_ms:number,error:Error?,usage:object?}.	Typed records conforming to versioned input/output contracts.	Internal node-contract call.	C5 rejects input/config mismatch or returns normalized operation error; C4 marks the node/run failed and skips dependent nodes.
I9 · C5 → C6	workspace_id:UUID, connection_id:UUID, capability:string, input:ValueMap, settings:object, timeout_ms:integer.	outputs:ValueMap; metadata: {provider:string,latency_ms:number,usage:object?}; error:Error?.	Provider-independent typed request/reply.	Internal capability call.	C6 rejects missing/out-of-scope connection and normalizes provider error/timeout; C5 returns a NodeResult error to C4.
I10 · C6 → E2	provider_model:string, payload:object, settings:object; server-only workload_auth:credential or temporary token.	provider_output:object; usage:object?; provider_request_id:string?; upstream_error:object?.	Provider-specific JSON; text/binary formats only for later multimodal adapters.	HTTPS provider API; configured HTTP loopback only for local models.	C6 handles 401/403, quota/429, 5xx, malformed output and timeout; no automatic retry of operations with possible side effects.
I11 · C6 → C7	workspace_id:UUID; connection_id:UUID?; operation:save load; metadata: {provider:string,capabilities:string[],auth_type:string,credential_ref:string}.	connection: {id:UUID,workspace_id:UUID,provider:string,auth_type:string,credential_ref:string}; commit_ok:bool.	Typed metadata rows only; credential_ref is a locator, never a raw secret.	Parameterized SQL request/reply; TLS hosted.	C7 reports missing connection/write failure; C6 rejects scope mismatch and returns safe connection error; no service call with unresolved credentials.
I12 · C4 → C7	execution_id:UUID?,workflow_id:UUID,workspace_id:UUID,graph_snapshot:Graph,status:RunStatus,node_results:NodeResult[], timestamps:timestamp[].	execution: {id:UUID,status:RunStatus,node_results:NodeResult[]}; commit_ok:bool.	Execution/node rows; snapshot and result values as JSON; redacted errors.	Parameterized SQL request/reply; TLS hosted.	C7 rolls back failed writes; C4 stops further dispatch when recording fails and returns storage error. Recovery marks unfinished runs interrupted; no blind replay.
I13 · C3 → C6	authorized_scope: {user_id:UUID,workspace_id:UUID}; provider:string; auth_type:string; credential_ref:string; display_name:string; capabilities:string[].	connection_id:UUID; provider:string; capabilities:string[]; validation_errors:Error[].	Typed connection registration; browser receives metadata only.	Internal application call after I5 manage_connection approval.	C6 rejects unsupported provider or invalid server-managed reference; C3 returns 422/403/503. C7 write failure prevents successful registration.

4. Interface specification table. (types and example)

Shared types: UUID = application identifier string; timestamp = UTC ISO 8601 string; object = structured JSON-compatible map; ? = optional/nullable; [] = array. Interfaces never expose provider access tokens or credential references to C1.

Type	Fields / meaning
User	id:UUID, display_name:string?, primary_email:string, avatar_url:URL?. Email is available to authenticated code; the current dashboard displays name/avatar.
WorkflowDraft / WorkflowRecord	WorkflowDraft = {name:string, graph:Graph?}. WorkflowRecord adds id:UUID, workspace_id:UUID, updated_at:timestamp. US2 may start with a temporary name-only object; graph is a later extension.
Graph	schema_version:integer; name:string; nodes:Node[]; edges:Edge[]. Node = {id:string,type:string,config:object,position:{x:number,y:number}}. Edge = {source:string,target:string,source_port:string,target_port:string}.
ValueMap	Map of port names to {type:text json, value:string object array number bool null}. Later media values use typed artifact references; they are outside the first slice.
NodeResult	node_id:string, status:RunStatus, outputs:ValueMap, latency_ms:number, error:Error?, usage:object?. RunStatus = pending running succeeded failed interrupted.
Error	code:string, message:string (safe for users), field:string?, node_id:string?, retryable:bool. Logs/results omit secrets; sensitive input/output content requires workspace access.

One short example payload — I4 workflow creation (US2)

POST of the following JSON creates a placeholder workflow record. The session cookie and CSRF header accompany it separately. A graph is optional at the US2 foundation stage.

```
{
  "workspace_id": "11111111-1111-4111-8111-111111111111",
  "workflow": {"name": "Example workflow", "graph": null}
}
```

Example reply: {"id": "22222222-2222-4222-8222-222222222222", "workspace_id": "11111111-1111-4111-8111-111111111111", "name": "Example workflow", "graph": null, "updated_at": "2026-10-02T16:00:00Z"}. This represents a saved placeholder, not a completed execution.

Data rules: Store internal users, identity associations, session digests, workspace membership, versioned graphs, connection metadata, execution snapshots, and node results. Retention durations and artifact storage are D1 decisions. Keep runtime credentials in server-managed configuration/secret storage, using references in C7. A user’s identity-provider login is separate from permission to call an AI service.

5. Data-flow diagram.

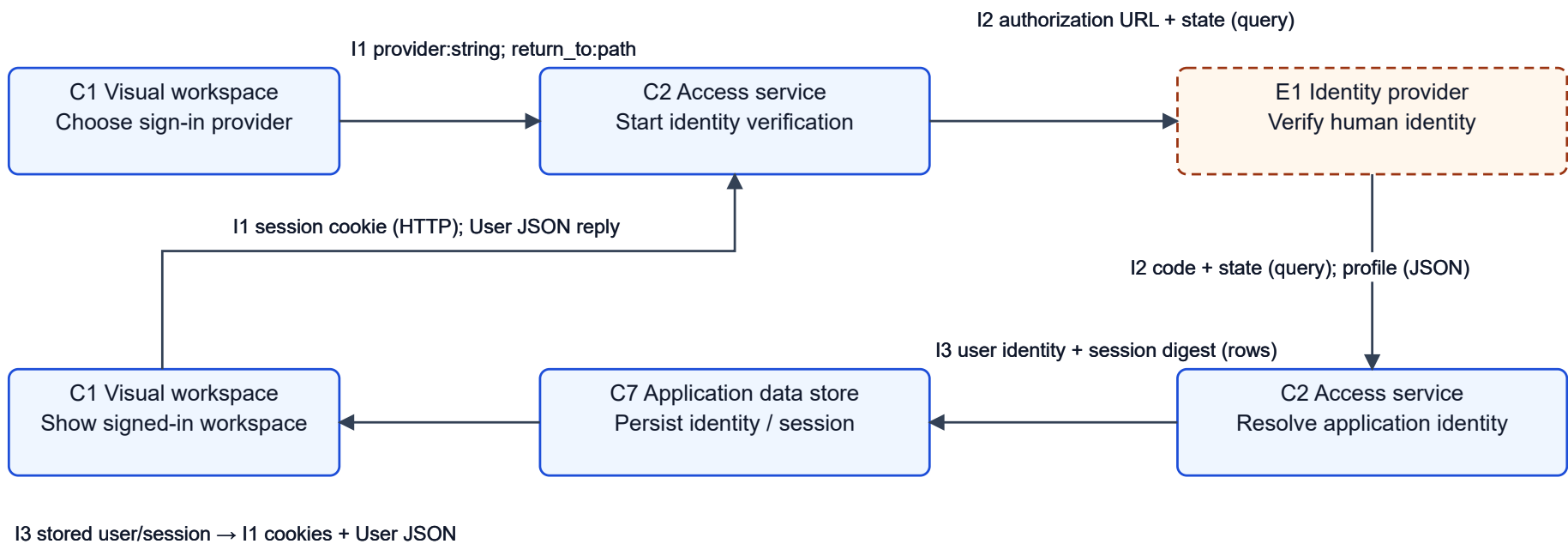
Flow A — human sign-in and session use

AI Workflow Studio — Design D0 | Data flow — access

Goal: Visually compose reusable AI nodes, run workflows across providers, and inspect their results.

Legend: solid blue box = team-built component/stage; dashed amber box = external system.

Arrows show request/data direction; replies return over the same interface. Labels identify data or interface IDs.



Stages may repeat a component. OAuth browser redirects are mediated by C1 through I1; provider exchanges belong to I2. I3 acknowledgments return to C2 before C2 issues I1 cookies. Session cookies are opaque; provider secrets stay server-side. Existing authentication limits: OAuth attempt 10 minutes; application session 7 days. These are lifetimes, not response SLAs.

5. Data-flow diagram. (continued)

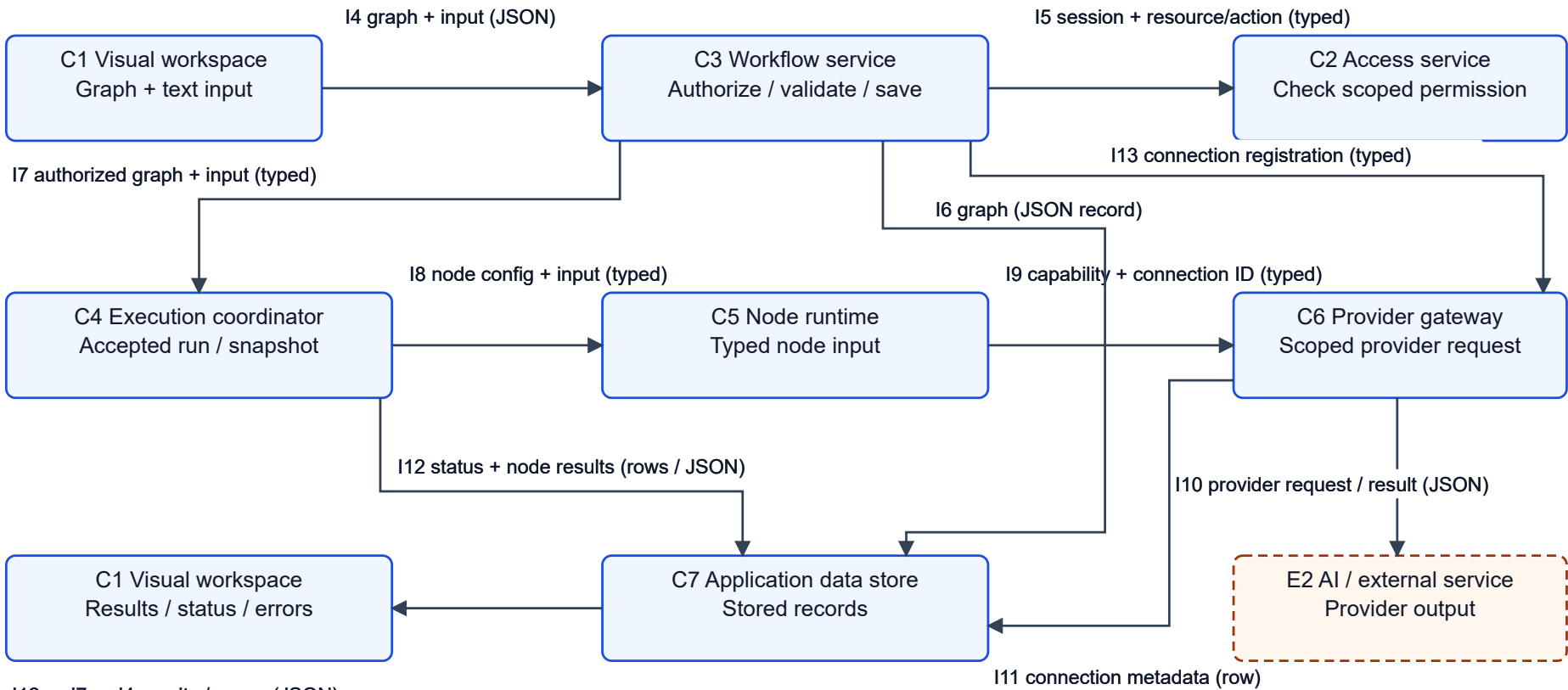
Flow B — compose, validate, execute, and inspect

AI Workflow Studio — Design D0 | Data flow — workflow

Goal: Visually compose reusable AI nodes, run workflows across providers, and inspect their results.

Legend: solid blue box = team-built component/stage; dashed amber box = external system.

Arrows show request/data direction; replies return over the same interface. Labels identify data or interface IDs.



I12 → I7 → I4 results / errors (JSON)

Replies: E2 → C6 → C5 → C4 over I10/I9/I8; C4 → C3 → C1 over I7/I4. C7 replies return to the requesting module.

Invalid graph → I4 field errors; denied access → I5 denial / I4 403; provider failure → normalized node error / stored failed run.

This is the future execution flow. US2 currently requires placeholder CRUD; User_stories.md sets no numerical SLA.

5. Data-flow diagram. (timing and failure paths)

US2's foundation flow is CRUD: JSON request → C3 → I6 → C7 stored record → JSON reply over I4. The diagrams show the existing access flow and the later product execution flow. Human identity and provider workload credentials remain separate.

Stage	Data transformation / outcome
US2 placeholder CRUD	Create/read/update/delete operates on a temporary WorkflowDraft/WorkflowRecord through I4/I6. Failed transactions leave the previous record intact; replies acknowledge persisted changes. Execution is later scope.
Compose → validate	C1 sends graph JSON through I4. C3 obtains I5 approval and validates node/edge contracts. Invalid graphs produce Error[] with node/field locations; no execution starts.
Validate → save	C3 stores an accepted graph through I6. Only a successful transaction produces a saved workflow identifier. The saved graph references connections by ID.
Save → run	C3 passes an authorized, immutable graph snapshot with typed runtime values through I7. C4 stores an execution through I12 before acknowledging acceptance.
Run → node → provider	C4 sends typed inputs through I8. C5 uses I9 only for provider-backed nodes. C6 loads workspace-scoped connection metadata via I11 and calls E2 via I10 with server-only credentials.
Provider → result → inspection	Provider JSON becomes normalized typed output over I10/I9/I8. C4 records status/output through I12. C1 reads status/results over I4/I7; readers receive only data permitted by I5.
Connection setup	C1 submits non-secret provider metadata through I4. C3 checks manage_connection through I5, then registers via I13/I11. Server-side provisioning supplies the credential reference.
Failure or interruption	Provider/node failure produces a normalized Error, a failed node/run, and skipped dependents. Recording failure halts dispatch. After process restart, incomplete records become interrupted; side-effecting calls are never silently replayed.

5. Data-flow diagram. (timing)

Timing budget and operating assumptions

Requirement evidence: User_stories.md contains no numeric latency, throughput, or deadline requirement; neither do the supplied assignment, project context, or Week 3 constraints. The limits below are proposals for later execution, not acceptance criteria added to US1–US5.

Item	D0 proposal / evidence	Effect on design
Identity lifetimes	Existing configuration: 10-minute OAuth attempt; 7-day fixed session.	Reject expired sign-in attempts or sessions; these are authentication lifetimes, not performance targets.
Provider / node call limit	Proposed initial 60-second maximum per external call; confirm against the chosen provider/node in D1.	C6 enforces a finite timeout and C4 records failure rather than waiting forever. This does not guarantee upstream cancellation.
Execution inspection	Proposed status polling every 2 seconds while the run view is open.	The browser uses I4/I7; push delivery can be added if measured demand warrants it.
Run acceptance	Proposed warm-environment target: acknowledge a recorded run within 2 seconds, excluding execution time.	Measure before claiming compliance; cold starts may exceed the target. Hosted backend remains active while a run executes.
Initial load	Low-traffic development use; no hard real-time or special hardware requirements documented.	Run on ordinary developer machines; postpone distributed execution and measure concurrency limits in D1.

6. Architecture pattern and justification.

Client-server governs C1's interaction with the backend through I1/I4. **Layered architecture** governs the single backend application: access/workflow services (C2/C3) coordinate execution (C4), node operations (C5), provider integration (C6), and persistence (C7). **Pipeline** governs data movement through the workflow graph: typed outputs become downstream typed inputs. Branches may be modeled in the graph, while parallel/loop execution remains later scope.

The deployment is a **modular monolith**: logical boundaries stay distinct without separate network services for every component. C7 persists shared authoritative state; external services remain outside that deployment.

Class criterion	Why this combination fits	Constraints / tradeoff
Fit to the problem	Client-server supports an interactive visual editor; a pipeline matches reusable connected nodes; layers separate graph policy from provider details.	The graph can span providers. Generic execution must not assume Azure, AWS, GCP, or OpenAI-specific behavior.
Team skills	Biographies show full-stack/UI/API experience (Ashanth, Daniel), cloud security/data pipelines (Khiem), and LLM evaluation (Jayrajsinh). Logical owners align work with those skills.	One backend reduces operational overhead for four students. Existing project decisions supersede the older constraints essay's tentative Angular/JWT choices; exact libraries are deferred to D1/D2.
Performance and timing	Internal calls reduce network overhead; acceptance and status are separate from slow model execution; finite timeouts make failures visible.	No hard timing requirement is documented. External-provider latency dominates. Local/in-process runs require explicit interruption handling and hosted compute that remains active during execution.
Scalability	Stateless browser requests plus persisted graphs/results allow the backend to scale later. C4/C6 boundaries can support workers/provider adapters when measurements justify them.	Multi-process execution ownership and durable job distribution are later design work. The first version must not promise distributed execution or automatic recovery.
Hardware constraints	Browser clients and normal developer computers suffice; cloud providers perform expensive inference. The local-model option uses available hardware.	No sensors or actuators are required. Local GPU inference is optional and hardware-dependent; the platform cannot assume every developer owns a GPU.

Week 3 constraints applied

Budget/hosting: Begin with reproducible local deployment and low-traffic hosted development. Avoid a separate service or database for each node; use one initial provider and bounded call sizes to limit testing expense. Compare actual hosting costs in D1 rather than claiming free operation.

Privacy/security: Enforce workspace authorization server-side, persist opaque session digests, separate human identity from workload credentials, and keep secrets out of workflow JSON and logs. Users must know that selected provider-backed nodes send their runtime input outside the application; retention and sensitive-data policy need explicit team decisions in D1.

Accessibility/social: The editor reduces orchestration code. Specify readable labels, textual validation errors, status text beyond color, and keyboard-accessible alternatives to drag-only editing. These are proposed usability requirements, not verified current features.

Rejected pattern: Microservices for every major component would add deployment, network-failure, and consistency work before traffic justifies it. Embedded sensor-actuator architecture does not fit a browser-based workflow platform without physical hardware.

6. Architecture pattern and justification. (foundation stories)

The foundation stories describe development and deployment work around the runtime diagram. Hosting services provide the deployment boundary; CI, documentation, and schema planning support delivery without becoming extra runtime interfaces.

Story	Required architecture / delivery work	D0 interpretation and verification to carry into D1
US1 — Azure environment	Developer Azure accounts; resource group; UI/API app registrations; Static Web Apps UI; App Service API; optional Key Vault for OAuth credentials.	Deploy a browser client plus one backend with persistent database connectivity. Check a full UI-to-API-to-database path and hosted authentication configuration; keep identity secrets server-side. These are verification checkpoints derived from the stated end-to-end testing use case.
US2 — Workflow CRUD	A placeholder API object with database-backed create, read, update, and delete.	I4/I6 explicitly include all CRUD operations. Start with a temporary workflow schema, then extend attributes/validation later. Check a create→read→update→delete round trip; do not require node execution to complete this story.
US3 — Database architecture	Tables, fields, connecting foreign keys, and relationship cardinalities.	C7's model includes users/identities/sessions, workspace memberships, workflows, provider connections, executions, and node results. Separate current foundation tables from later target tables; define deletion and retention policies in D1.
US4 — CI pipeline	Repository health checks, automated tests/lint, and protection of main.	Keep checks in .github/workflows and configure required checks in repository settings. Verify checks run on proposed changes and failures prevent merging when branch protection is configured. Deployment automation is not explicitly required by this story.
US5 — Platform fundamentals	Vite frontend with Angular or React; FastAPI/virtual environment; documentation folder containing Markdown.	Use the existing React/FastAPI foundation. Check reproducible setup and retain Markdown context plus this design's editable sources. The story permits React; it does not require an Angular migration.

Hosting discrepancy to resolve: US1 specifies **Azure App Service** for the API, while Docs/Project.md proposes **Azure Container Apps**. D0 keeps one logical backend and records both source statements; it does not treat the choice as approved. Confirm the hosting service before implementing hosted deployment. Database hosting is also not selected by the supplied stories.

Optional secret storage: US1 makes Key Vault optional. This design requires safe server-side credential handling; it does not make Key Vault a mandatory external dependency. Add its concrete interface in D1 if the team selects it.

7. Decision log.

These proposed D0 decisions capture rationale for team review and reuse in D2 and the Week 9 technical specification. They do not claim decisions were approved in a team meeting.

ID	Decision	Alternatives considered	Why the chosen option wins / consequence
D01	Use client-server + layers + workflow pipeline inside a modular monolith.	Microservices per component; all logic in the browser; one undifferentiated backend.	Matches editor-to-engine data flow and a small team's operating capacity, while preserving boundaries for later extraction.
D02	Use server-side opaque sessions for browser authentication.	Browser-held bearer tokens; local username/password accounts.	Matches implemented authentication and central revocation. Browser cookie requests require CSRF controls and secure hosted transport.
D03	Separate human identity from workspace provider connections.	Reuse login-provider tokens; embed API keys in node configuration.	Prevents login from silently granting cloud permissions and keeps shared workflow graphs free of secrets; provider onboarding is an independent task.
D04	Keep the execution coordinator provider-independent through typed nodes and a provider gateway.	A separate engine per cloud; provider-specific branching in C4.	Allows a graph to mix providers and reusable operations. C6 contains capability/authentication differences and normalizes upstream errors.
D05	Persist versioned workflow snapshots and execution/node records.	Keep results only in browser memory; overwrite the current graph during a run.	Users can inspect what actually ran despite later edits. Persistence is required before accepting a run; retention and storage size require limits in D1.
D06	Start with low-volume in-process execution plus explicit interruption status.	Distributed workers/message broker now; long blocking browser request; silent replay after a crash.	Keeps early deployment small and lets the UI inspect status separately. Interrupted/side-effecting runs need deliberate user action rather than blind replay.
D07	Use one relational application data store with structured graph/result fields.	Separate database per component; schema-free store for all records.	Users, memberships, workflows, and executions have ownership relationships that need consistent updates. Structured fields retain flexible node contracts.
D08	Complete foundation stories before the text/JSON execution demonstration.	Require a full graph engine for placeholder CRUD; implement all providers/media immediately.	US1–US5 establish hosting, CRUD, schema, CI, and scaffolding. A text/JSON workflow with one provider is the next product target; execution needs dedicated future stories.